

PLATEFORME DE PARTAGE DE CODE

Livrable S2 — Mini-CDC · WBS · Estimation · Gantt

Méthodologies de Développement Logiciel

Pr. Sanae EL MIMOUNI | Faculté des Sciences, Université Mohammed V de Rabat

Année universitaire 2025–2026

Projet	Plateforme de partage de code
Équipe	Équipe N° 4
Membres	Khaoula Lakhel Zineb AIT OULAHSEN Aya Harrati

Table des matières

1. Mini-Cahier des Charges (Mini-CDC)	3
1.1 Contexte	3
1.2 Parties prenantes	3
1.3 Périmètre fonctionnel (MoSCoW)	3
1.4 Contraintes	3
1.5 Dépendances externes	3
1.6 Cas d'utilisation	4
1.7 Architecture technique	5
1.8 Risques	5
1.9 Glossaire	5
1.10 Hypothèses et exclusions	5
1.11 KPIs	5
1.12 Planification macro	6
1.13 Critères de succès	6
1.14 Wireframes	6
2. Work Breakdown Structure (WBS)	8
2.1 Arbre WBS (3 niveaux)	8
2.2 Dictionnaire WBS	9
3. Estimation du projet	9
3.1 Méthode par Planning Poker	9
4. Diagramme de Gantt	9
4.1 Lister et estimer les tâches	9
4.2 Tableau des marges	10
4.3 Diagramme de Gantt	10

1. Mini-Cahier des Charges

Mini-CDC — Définition fonctionnelle du projet

1.1 Contexte

Les étudiants en informatique rencontrent des difficultés en programmation et utilisent souvent des plateformes générales pour échanger et corriger leurs erreurs. Ils ont donc besoin d'un espace dédié favorisant la collaboration et l'accompagnement par des professeurs ou des experts. Notre plateforme permet de réduire le temps de recherche de solutions de 30 % à 50 %, tout en favorisant les échanges entre étudiants, programmeurs et experts du domaine.

1.2 Parties prenantes

Rôle	Besoins	Implication
Étudiants	Partager et consulter des codes, apprendre	Forte
Administrateur	Gérer utilisateurs, contenus et sécurité	Forte
Développeur	Conçoit, développe et maintient l'application	Forte
Professeurs	Suivre les activités, guider les étudiants	Moyenne
Programmeurs	Partager leur expérience et aider	Moyenne

1.3 Périmètre fonctionnel (MoSCoW)

	Fonctionnalité	Description courte	Priorité
F1	Inscription et connexion des utilisateurs	Créer un compte et se connecter	Must
F2	Publication de code	Publier des solutions de programmation avec un titre, une description et le langage utilisé	Must
F3	Commentaires sur les codes	Pour signaler des erreurs, poser des questions ou proposer des améliorations	Should
F4	Recherche	Permet de rechercher un code ou un algorithme précis	Could
F5	Menu	Permet de trier l'ensemble des codes partager selon le langage de programmation utilisé (C, C++, JAVA, ect)	Won't

1.4 Contraintes

- Delai : 3mois.
- Equipe : 2 développeurs.

1.5 Dépendances externes

API tierce : Authentification via compte GitHub.

Risque si indisponible : Les utilisateurs ne peuvent pas se connecter via GitHub.

Systèmes existants : Authentification université, connexion avec email universitaire.

Risque si indisponible : Impossible de se connecter.

1.6 Cas d'utilisation

Scénario 1 : Inscription / Connexion :

Acteur	Utilisateur (étudiant, professeur ou programmeur)
Précondition	• L'utilisateur possède un compte • Il accède à la page de connexion
Déroulement	1. L'utilisateur saisit son email 2. Il saisit son mot de passe 3. Il clique sur le bouton "Se connecter" 4. Le système vérifie les informations 5. Si les informations sont correctes, l'utilisateur est connecté
Résultat	L'utilisateur accède à son compte et à la plateforme
Cas d'erreur	• Si le mot de passe est incorrect, le système affiche un message d'erreur : "Mot de passe incorrect" et refuse l'accès • Si l'email n'existe pas, un message indique "Compte introuvable"

Scénario 2 : Publier un code

Acteur	Étudiant
Précondition	L'étudiant est inscrit et connecté à la plateforme
Déroulement	1. L'étudiant clique sur "Publier un code" 2. Il remplit le titre, choisit le langage et saisit son code 3. Il ajoute une description 4. Il clique sur "Publier" 5. Le système enregistre et affiche le code
Résultat	Le code est publié et visible par tous les utilisateurs
Cas d'erreur	Titre ou code vide → message d'erreur "Champs obligatoires manquants"

Scénario 3 : Commenter un code

Acteur	Étudiant, Expert ou Professeur
Précondition	L'utilisateur est connecté et consulte la page d'un code existant
Déroulement	1. L'utilisateur lit le code publié 2. Il saisit son commentaire dans le champ prévu 3. Il clique sur "Envoyer" 4. Le système enregistre et affiche le commentaire
Résultat	Le commentaire apparaît sous le code, visible par tous

Cas d'erreur	Commentaire vide → message d'erreur "Le commentaire ne peut pas être vide"
---------------------	--

1.7 Architecture technique

Langages / Frameworks	Frontend: HTML/CSS/JavaScript Backend: Node.js / Express
Base de données	MySQL
Hébergement	GitHub Pages
Contraintes sécurité	HTTPS.

1.8 Risques

	Description du risque	Probabilité	Mitigation
R1	Les utilisateurs peuvent publier des codes incorrects, inutiles ou du spam.	Élevé	Ajouter un système de commentaires
R2	Difficulté à implémenter certaines fonctionnalités (Backend)	Moyen	Utiliser des technologies simples (Node.js + Express)
R3	L'application peut devenir lente si beaucoup de données (codes, commentaires).	Moyen	Optimiser les requêtes

1.9 Glossaire

1. **Snippet** : Bloc de code publié avec un titre, un langage et une description.
2. **Utilisateur** : Toute personne inscrite — étudiant, professeur ou expert.
3. **Commentaire** : Réponse publiée sous un snippet pour corriger, questionner ou améliorer.
4. **Langage** : Langage de programmation du snippet (Java, Python, C++...).
5. **Administrateur** : Utilisateur avec droits étendus pour gérer et modérer la plateforme.
6. **Expert** : Utilisateur reconnu pour apporter des réponses techniques validées.
7. **Publication** : Action de soumettre un snippet visible par toute la communauté.
8. **Fil de discussion** : Snippet + ses commentaires formant un échange collaboratif.

1.10 Hypothèses et exclusions

- a) **Hypothèses** :
- Les utilisateurs ont Internet
 - Les étudiants savent utiliser une plateforme web
- b) **Exclusions** :
- Pas d'intégration IA avancée

1.11 KPIs

- Nombre d'utilisateurs inscrits (environ 15 étudiants)
- Nombre de codes publiés (environ 10 codes sera publié)
- Nombre de commentaires (environ 7 commentaires)
- Temps de réponse aux questions (24h au moins)
- Fonctionnalités principales opérationnelles (100%)

1.12 Planification macro

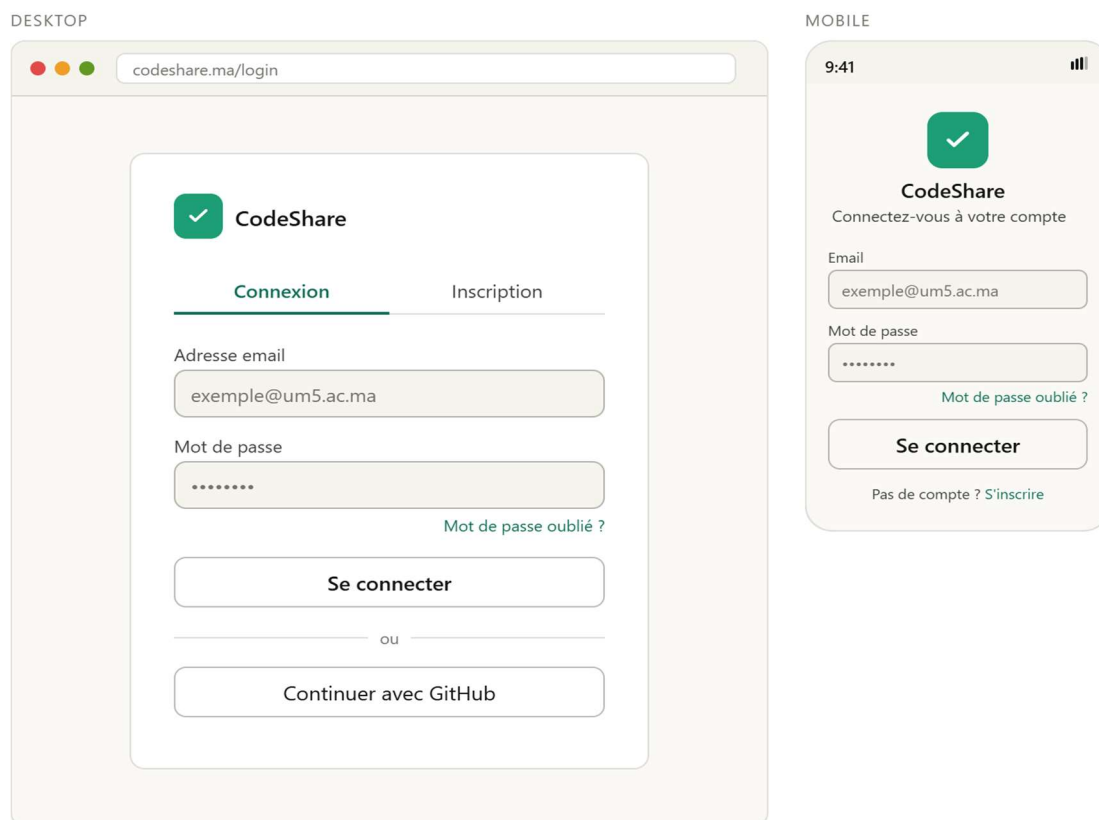
	Livrable attendu	Date estimée	Responsable
S1	Fiche Projet (ce document)	Avant S2	Chef de projet
S2	Diagramme de Gantt	Avant S3	Chef de projet
S3	Réseau PERT	Avant S4	Chef de projet
S5	Product Backlog v1	Avant S6	Chef de projet
S6	Sprint Backlog + Sprint 0	Avant S7	Chef de projet
S7	Gestion Jira (TP1)	Avant S8	Chef de projet
S9	Dépôt Git structuré	Avant S10	Chef de projet
S10	Pipeline CI	Avant S12	Chef de projet
S12	Finale	Avant S13	Toute l'équipe

1.13 Critères de succès

- L'application est pleinement fonctionnelle (75%)
- Les 2 fonctionnalités Must Have sont opérationnelles (100%)
- L'interface est claire et utilisable (60%)
- Aucun bug critique
- Démonstration réussie lors de la soutenance (80%)

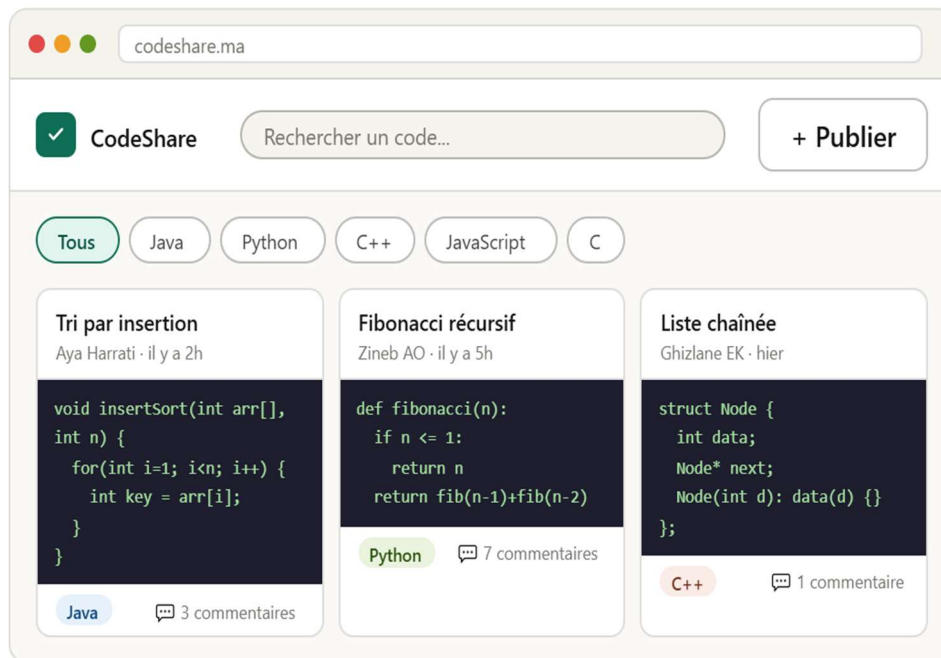
1.14 Wireframes

a) La page Inscription / Connexion :

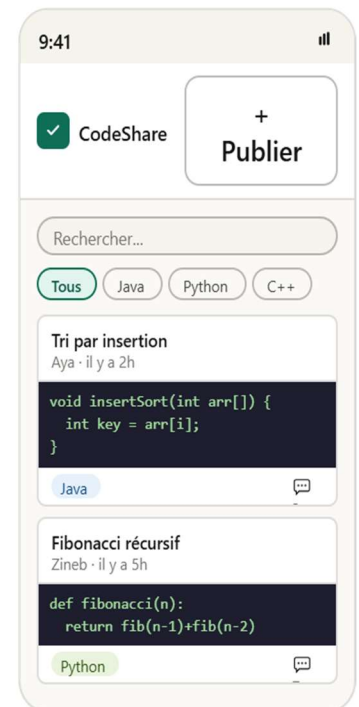


b) La page d'accueil :

DESKTOP

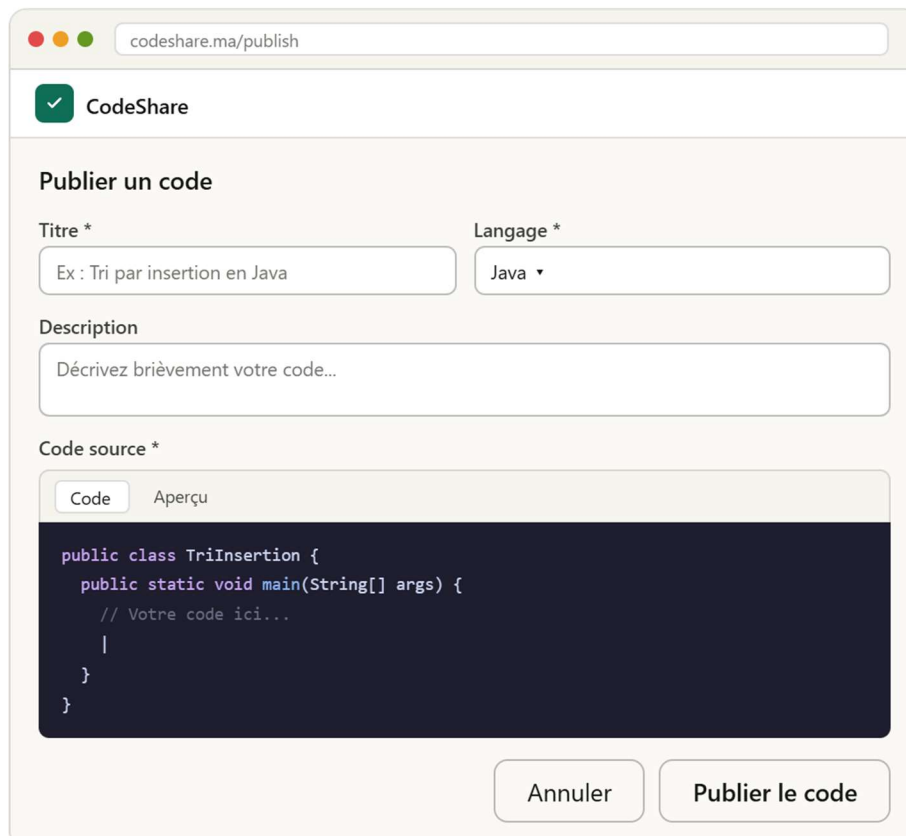


MOBILE



c) La page de publier un code :

DESKTOP



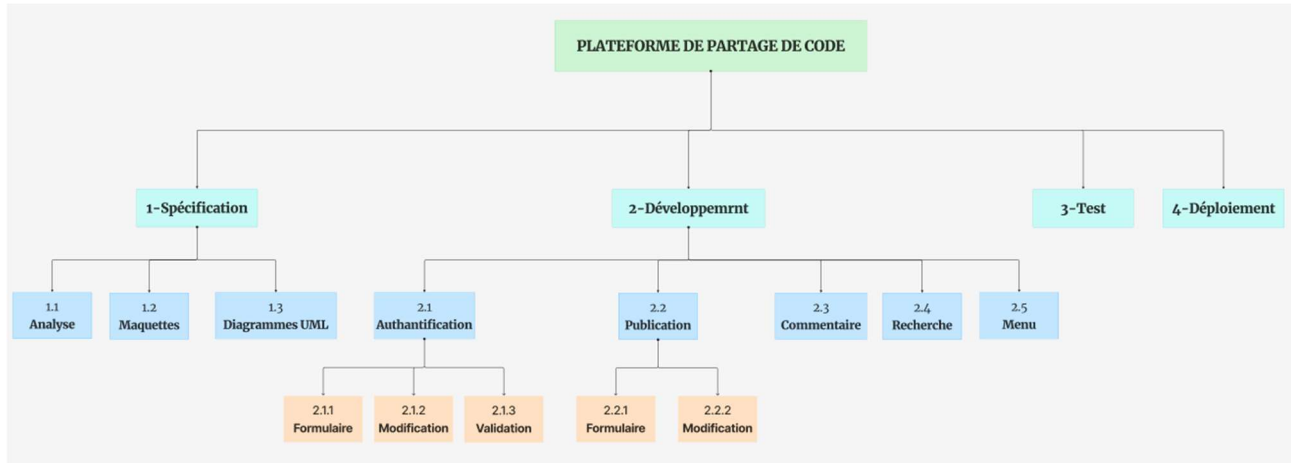
MOBILE



2. Work Breakdown Structure (WBS)

Décomposition hiérarchique du projet en tâches élémentaires

2.1 Arbre WBS (3 niveaux)



2.2 Dictionnaire WBS

Code	Livrable	Responsable	Durée estimée	Critère de fin
1.1	Cahier d'analyse des besoins validé	Chef + Client	5j	Signé par MOA
1.2	Maquettes validées par le client	Designer	1j	Validées client
1.3	Diagrammes UML	Architecte	2j	Validés par les Développeurs
2.1.1	Formulaire authentification opérationnel	Dev1	1j	Interface du formulaire apparaitre
2.1.2	Fonction de validation opérationnelle	Dev1	3j	Authentification vérifiée
2.1.3	Mise à jour de la table utilisateur de la BD	Dev1	3j	Modification validé et connexion
2.2.1	Formulaire de publication fonctionnel	Dev2	2j	Apparition de l'interface de publication de code
2.2.2	Mise à jour de la BD	Dev2	3j	Modification validé et code publier
2.3	Section des commentaires opérationnelle	Dev1	2j	Commentaire publier
2.4	Bare de recherche opérationnelle	Dev2	2j	Recherche validé
2.5	Menu opérationnel	Dev1	2j	Tri validé
3	Rapport de test	Tous les développeurs	5j	Couverture 70%
4	Support de formation utilisateur validé	Chef de projet	2j	Validé par administrateurs

3. Estimation du projet

Méthodes d'estimation : Analogie

3.1 Méthode par Analogie

Le projet référence c'est GitHub, **justification** : GitHub est un projet de partage de code connu et complet. Il inclut pratiquement toutes les fonctionnalités qu'un projet de dépôt de code pourrait avoir : publication de code, authentification, commentaires, gestion de versions, Pull Requests, CI/CD, wiki, issues, etc. Ça en fait un bon projet "référence" pour mesurer ce qui est plus simple ou plus complexe dans notre projet de plateforme de partage de code.

Différence identifiée	Impact	Coefficient	Pourquoi?
Menu personnalisé	Périmètre plus grand	1,10	Nouvelle interface, effort supplémentaire, mais simplifié pour tenir le temps
Issues / Suivi des bugs	Périmètre plus petit	0,65	Simplifie le projet
Pull Requests / Gestion de versions	Périmètre plus petit	0,60	Pas de collaboration avancée
Wiki / Documentation complète	Périmètre plus petit	0,65	Documentation simplifiée
Actions / Intégration continue (CI/CD)	Périmètre plus petit	0,65	Pas d'automatisation des workflows

→ Calcule de temp de développement pure :

6 mois (GitHub) $\times 1.10 \times 0.65 \times 0.60 \times 0.65 \times 0.65 \approx 1.087$

→ Résultat ≈ 4 à 5 semaines.

- **Conclusion :**

Résultat acceptable par rapport à notre délai 3mois.

4. Diagramme de Gantt

Planification des tâches dans le temps

4.1 Lister et estimer les tâches

ID	Nom de la tâche	Responsable	Prédécesseur	Durée
A	Analyse et CDC	Chef + Client	---	5j
B	Conception	Architecte	A	3j
C	Authentification	Dev1	B	1semaine
D	Publication	Dev2	B	5j
E	Commentaire	Dev 1	B, C	2j
F	Recherche	Dev2	B, D	2j
G	Menu	Dev2	B, D, F	2j
H	Test	Testeur	C, D, E, F, G	5j
I	Déploiement	Chef de projet	H	2j

4.2 Tableau des marges

$MT = \text{Début au plus tard} - \text{Début au plus tôt} \mid MT = 0 \rightarrow \text{tâche critique}$

Tâche	Durée	Début + tôt	Début + tard	Marge totale	Critique ?
A- Analyse et CDC	5j	Jour 1	Jour1	0	Oui
B- Conception	3j	Jour 6	Jour 6	0	Oui
C- Authentification	7j	Jour 9	Jour 30	3 semaines	Non
D- Publication	5j	Jour 9	Jour 32	23 jours	Non
E- Commentaire	2j	Jour 16	Jour 35	19 jours	Non
F- Recherche	2j	Jour 14	Jour 35	3 semaines	Non
G- Menu	2j	Jour 14	Jour 35	3 semaines	Non
H- Test	5j	Jour 37	Jour 37	0	Oui
I- Déploiement	2j	Jour 42	Jour 42	0	Oui

4.3 Diagramme de Gantt final

